
SQL Note: SUBQUERY, GROUP BY, ORDER BY, HAVING, LIMIT

◆ 1. SUBQUERY

A **Subquery** is a query nested inside another query. It can be used in `SELECT`, `FROM`, or `WHERE` clauses.

◆ Syntax:

```
SELECT column1 FROM table1
WHERE column2 = (SELECT column2 FROM table2 WHERE condition);
```

◆ Example:

```
-- Get names of employees who earn more than the average salary
SELECT name FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

◆ 2. GROUP BY

`GROUP BY` groups rows with the same values in specified columns and allows aggregate functions like `SUM()`, `AVG()`, `COUNT()` to be applied.

◆ Syntax:

```
SELECT column1, AGG_FUNC(column2)
FROM table
GROUP BY column1;
```

◆ Example:

```
-- Total salary per department
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department;
```

◆ 3. ORDER BY

ORDER BY is used to sort the result set by one or more columns, either ascending (ASC) or descending (DESC).

◆ Syntax:

```
SELECT column1, column2
FROM table
ORDER BY column1 ASC|DESC, column2 ASC|DESC;
```

◆ Example:

```
-- List all employees by descending salary
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

◆ 4. HAVING

HAVING is like WHERE, but it's used **after GROUP BY** for filtering aggregated data.

◆ Syntax:

```
SELECT column1, AGG_FUNC(column2)
FROM table
GROUP BY column1
HAVING AGG_FUNC(column2) condition;
```

◆ Example:

```
-- Departments with total salary more than 100000
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 100000;
```

◆ 5. LIMIT

LIMIT restricts the number of rows returned in the result.

◆ Syntax:

```
SELECT * FROM table  
LIMIT number;
```

◆ Example:

```
-- Show top 5 highest paid employees  
SELECT name, salary  
FROM employees  
ORDER BY salary DESC  
LIMIT 5;
```

◆ Bonus: Combine Them Together

```
-- Top 3 departments with highest total salary, only if their total exceeds  
100000  
SELECT department, SUM(salary) AS total_salary  
FROM employees  
GROUP BY department  
HAVING SUM(salary) > 100000  
ORDER BY total_salary DESC  
LIMIT 3;
```

Tip:

- Use `WHERE` for filtering **before grouping**.
- Use `HAVING` for filtering **after grouping**.
- Subqueries can be used for dynamic filtering, comparisons, or temporary data sets.



Scenario-Based Subquery Questions Set



Scenario:

You're working with a database for a company named **BrightTech Solutions**. There are two tables:

◆ employees

emp_id	name	department	salary	manager_id
--------	------	------------	--------	------------

1	Ahmed	IT	70000	5
2	Rina	HR	55000	6
3	Karim	IT	80000	5
4	Fatima	Finance	60000	6
5	Saleh	IT	100000	NULL
6	Sara	HR	95000	NULL

◆ projects

project_id	project_name	budget	emp_id
------------	--------------	--------	--------

101	Alpha	500000	1
102	Beta	250000	2
103	Gamma	600000	3
104	Delta	300000	4
105	Omega	800000	5



Questions (With Subquery)

? Q1.

Find the names of employees who earn more than the average salary of all employees.

```
SELECT name
FROM employees
WHERE salary > (
    SELECT AVG(salary) FROM employees
);
```

? Q2.

List all employees who work on a project whose budget is greater than the average budget of all projects.

```
SELECT name
FROM employees
WHERE emp_id IN (
    SELECT emp_id
    FROM projects
    WHERE budget > (SELECT AVG(budget) FROM projects)
);
```

? Q3.

Show the name and salary of employees whose salary is more than their manager's salary.

```
SELECT name, salary
FROM employees AS e
WHERE salary > (
    SELECT salary
    FROM employees
    WHERE emp_id = e.manager_id
);
```

? Q4.

Find the employee(s) who work on the project with the highest budget.

```
SELECT name
FROM employees
WHERE emp_id = (
    SELECT emp_id
    FROM projects
    WHERE budget = (SELECT MAX(budget) FROM projects)
);
```

? Q5.

List names of employees who do not work on any project.

```
SELECT name
FROM employees
WHERE emp_id NOT IN (
    SELECT emp_id FROM projects
);
```

Perfect! Here's a **complete scenario** with **multiple SQL questions and solutions** covering:

- ◆ GROUP BY
 - ◆ ORDER BY
 - ◆ HAVING
 - ◆ LIMIT
-



Scenario: E-Commerce Company - ShopZone

You are working with a database for an online shopping platform **ShopZone**. There is a table called:

- ◆ **orders**

order_id	customer_name	category	amount	country
1	Ayesha	Electronics	500	Bangladesh
2	Rafiq	Clothing	150	India
3	Sohail	Electronics	700	Bangladesh
4	Nadia	Clothing	200	Bangladesh
5	Hasan	Electronics	1200	India
6	Rina	Toys	100	India
7	Zayed	Clothing	250	Bangladesh

order_id	customer_name	category	amount	country
8	Tania	Toys	350	India
9	Imran	Electronics	900	Bangladesh

Questions + SQL Solutions:

? Q1.

Find the total sales amount for each product category.

 *Topic: GROUP BY*

```
SELECT category, SUM(amount) AS total_sales
FROM orders
GROUP BY category;
```

? Q2.

List the total sales by country, sorted from highest to lowest.

 *Topics: GROUP BY + ORDER BY*

```
SELECT country, SUM(amount) AS total_sales
FROM orders
GROUP BY country
ORDER BY total_sales DESC;
```

? Q3.

Show product categories where total sales exceeded 1000.

 *Topics: GROUP BY + HAVING*

```
SELECT category, SUM(amount) AS total_sales
FROM orders
GROUP BY category
HAVING SUM(amount) > 1000;
```

? Q4.

Show the top 2 highest spending countries.

■ *Topics: GROUP BY + ORDER BY + LIMIT*

```
SELECT country, SUM(amount) AS total_spent
FROM orders
GROUP BY country
ORDER BY total_spent DESC
LIMIT 2;
```

? Q5.

Find the top 3 categories by average order amount.

■ *Topics: GROUP BY + ORDER BY + LIMIT*

```
SELECT category, AVG(amount) AS avg_order_amount
FROM orders
GROUP BY category
ORDER BY avg_order_amount DESC
LIMIT 3;
```

? Q6.

Which countries have an average order amount greater than 400?

■ *Topics: GROUP BY + HAVING*

```
SELECT country, AVG(amount) AS avg_order
FROM orders
GROUP BY country
HAVING AVG(amount) > 400;
```
